

TECHNICAL WHITE PAPER • MARCH 2026 • V1.0

WARD Protocol

Deterministic Default Protection for XRPL Lending

[Author](#)

Will Flores • wardprotocol.org

[Version](#)

SDK v0.1.1 • [PyPI](#)

[Status](#)

Testnet Proven • 5 On-Chain TXs

[Discussion](#)

[XRPLF #474](#)

[License](#)

[MIT Open Source](#)

CORE INVARIANT **ward_signed = False – always**

TABLE OF CONTENTS

01 [Abstract](#)

02 [Problem Statement](#)

03	Solution – Ward Protocol
04	Technical Architecture
05	9-Step Claim Validation
06	Security Model
07	Privacy Roadmap – XLS-96
08	Business Model
09	Proof of Implementation
10	Roadmap
11	Conclusion

SECTION 01

Abstract

Ward Protocol is an open-source, off-ledger infrastructure layer for deterministic default protection on XLS-66 lending vaults on the XRP Ledger. It is not an insurance company, not a custodian, and not an operator. It is a software specification – the same architectural model Stripe applies to payments, applied to institutional risk management on XRPL.

Ward defines how vault default protection should work. Institutions implement the specification using their own wallets, their own capital, and their own regulatory licenses. Ward writes the rules. Institutions run the rails. Everything settles on XRPL.

Core invariant: `ward_signed = False` – always. Ward constructs unsigned XRPL transactions. Institutions sign with their own wallets. XRPL settles. Ward's server is irrelevant once a transaction hits the ledger.

The XRPL Foundation Executive Director confirmed in March 2026 that the trust gap in XLS-66 vaults is real, that risk management infrastructure for XRPL DeFi is a legitimate and open gap, and that Ward's off-ledger API-first approach is the correct starting point.

SECTION 02

Problem Statement

XLS-66 lending vaults are live on XRPL. Institutions can lend. Depositors can earn. But if a vault defaults, there is no on-chain protection layer — no specification, no standard, no settlement rails.

The Trust Gap

XLS-66 vaults give institutions the ability to accept deposits and make loans on the XRP Ledger. But the standard provides no mechanism for what happens when a borrower defaults. Depositors who place capital into an XLS-66 vault have no trustless recourse if the loan manager behaves improperly or a borrower fails to repay.

This is not a theoretical concern. It is the primary barrier to institutional capital deployment into XRPL DeFi. Risk without recourse is not deployable capital.

Existing Solutions Are Insufficient

DeFi lending protocols across chains address defaults inconsistently — some rely on external oracles, some on governance votes, some on manual arbitration. None of these approaches are acceptable to institutional counterparties who require deterministic, auditable, legally-defensible outcomes.

- **Oracle-dependent systems** introduce third-party risk and manipulation vectors
- **Governance-based resolution** is slow, unpredictable, and unsuitable for time-sensitive defaults
- **Manual arbitration** is not scalable and introduces counterparty risk
- **No standard at all** — each platform builds custom, incompatible solutions

Ward Protocol introduces a single, auditable, deterministic standard that eliminates all four failure modes without requiring a new blockchain amendment, an external oracle, or a governance layer.

SECTION 03

Solution

Ward Protocol connects four existing XRPL standards into a coherent, deterministic default protection specification. No new chain. No oracle. No custodian. No governance.

STANDARD	ROLE	WARD'S CONTRIBUTION
XLS-66	Lending Vaults	Vault monitoring spec, default detection threshold (health ratio < 1.5)
XLS-80	Permissioned Domains	Compliant domain configuration for institutional vault access
XLS-70	On-Chain Credentials	KYC/AML credential schema with SHA-256 hash anchoring

XLS-20	NFT Policy Certificates	Policy metadata schema – taxon 282, tfBurnable, no tfTransferable
XRPL Escrow	Settlement	PREIMAGE-SHA-256 condition structure for claim settlement

Three Phases, All On-Chain

01	Setup — Institution calls	<code>register_vault()</code>	. Ward returns an unsigned	<code>AccountSet</code>	transaction. Institution signs and submits. Vault appears in XLS-80 domain registry on XRPL.
02	Coverage — Institution calls	<code>mint_policy_nft()</code>	. Ward returns an unsigned	<code>NFTokenMint</code>	with Ward's metadata schema encoded in the URI. Institution mints — it is their certificate, not Ward's.

03

Default & Settlement

WardMonitor

runs in the institution's infrastructure. On default detection (health ratio < 1.5, confirmed over 3 ledger closes), Ward builds an unsigned

EscrowCreate

. Institution signs. XRPL locks funds for 48 hours. Claimant finishes with PREIMAGE-SHA-256 fulfillment. NFT burns atomically – replay protection enforced by the ledger.

```
# Ward NEVER does this – Ward has no wallet await
submit_and_wait(tx, client, ward_wallet) # Ward ALWAYS does this
return UnsignedTransaction( tx_dict=tx.to_dict(), ward_signed=False
# always ) # Institution signs and submits with their own wallet
```

SECTION 04

Technical Architecture

Ward Protocol is an off-ledger API layer. The specification is implemented as a

Python SDK with five hardened modules, each responsible for a distinct phase of the protocol lifecycle.

Five Hardened SDK Modules

#	CLASS	PURPOSE
1	WardClient	Policy purchase — premium payment + NFT mint
2	VaultMonitor	WebSocket default detection, 3-ledger confirmation window
3	ClaimValidator	9-step adversarial-hardened on-chain validation
4	EscrowSettlement	PREIMAGE-SHA-256 conditioned claim settlement
5	PoolHealthMonitor	On-chain solvency monitoring and dynamic premium pricing

Architecture Layers

- **XRPL**

— authoritative state. XLS-66 vaults, XLS-20 NFT policies, XLS-70 credentials, XLS-80 domains, native escrow. Ward reads from this layer; institutions write to it.
- **Ward Protocol Layer**

— off-ledger validation engine, policy manager, unsigned transaction builder. Stateless by design. No database. No Ward-controlled keys.
- **Institution Layer**

— the institution's wallet signs transactions, runs

WardMonitor

in their own infrastructure, and holds regulatory licenses. Ward never has access to this layer.

Why off-ledger? The XRPL Foundation confirmed that most of Ward's logic — vault monitoring, risk scoring, claim validation — is better suited to an off-ledger API

layer. The existing Python API at api.wardprotocol.org is precisely the architecture recommended.

SECTION 05

9-Step Claim Validation

When a depositor files a default claim, Ward's `ClaimValidator` runs nine deterministic checks — all on-chain, no oracle, no human judgment, no governance. All nine must pass to produce a settlement transaction. Any single failure returns a reason code and rejects the claim.

- 01 Vault exists on ledger
- 02 Policy NFT valid and owned by claimant
- 03 Policy unexpired
- 04 Depositor address matches NFT
- 05 Collateral ratio breached (health ratio < 1.5)
- 06 No active escrow already pending
- 07 KYC credential (XLS-70) valid and unexpired
- 08 Domain credential (XLS-80) valid
- 09 No duplicate claim on this NFT

Replay protection: The XLS-20 policy NFT burns on settlement. Once burned, no

second claim is possible against the same policy — not by Ward, not by anyone.
Replay protection is enforced by the XRPL ledger itself.

Default Detection Threshold

WardMonitor tracks vault health ratio continuously via WebSocket connection to XRPL. A default is declared when:

- Health ratio drops below 1.5 (the Ward Protocol threshold)
- The condition persists across 3 consecutive ledger closes (~9–12 seconds)
- No active escrow is already in progress for this vault

The 3-ledger confirmation window prevents single-block manipulation and false positives from transient network conditions.

SECTION 06

Security Model

Eight security invariants — all enforced by the XRPL ledger, not Ward's server.

INVARIANT	ENFORCEMENT MECHANISM
Ward never holds wallet keys	By architecture — no wallet object exists in Ward's codebase
Policies non-transferable	<code>tfBurnable</code> only, no <code>tfTransferable</code> — XRPL enforces at protocol level
No front-running on claims	PREIMAGE-SHA-256 — only the claimant holds the

	fulfillment key
Clock manipulation impossible	XRPL ledger time used for all expiry logic – not system clock
Multi-confirmation before default	3 ledger closes required – no single-block manipulation possible
Replay attacks impossible	NFT burns on settlement – ledger confirms absence atomically
Rate limiting on claims	Max 3 attempts per NFT per 5-minute window
Address validation	All inputs validated with XRPL ledger codec before any transaction is constructed

For a complete analysis of all 15 attack vectors and their mitigations, see [security_notes.md](#) in the Ward Protocol repository.

SECTION 07

Privacy Roadmap – XLS-96

Ward Protocol's v2 privacy layer is fully XRPL-native – no external dependencies, no breaking changes to the existing specification.

- **XLS-96** – EC-ElGamal encryption with ZKPs for MPT balances. Vault
 - Confidential** share tokens (XLS-65) are a natural candidate for confidential
 - Balances** MPTs under XLS-96.
-

- **MPT** — view keys scoped to XLS-80 permissioned domain members.
- Selective** Regulators and institutional counterparties get auditability
- Disclosure** without public balance exposure.

The plaintext/confidential boundary is maintained at the settlement layer — XRPL escrow operates on plaintext XRP, which is required for settlement. Vault share tokens and pool balances are candidates for confidential treatment; loan and collateral objects remain plaintext for deterministic validation.

This gives institutional lenders the privacy model they actually need: internal balance confidentiality with regulatory auditability on demand — without any change to the core Ward invariant or settlement flow.

SECTION 08

Business Model

Ward Protocol generates revenue as a specification layer — not as an operator, custodian, or insurer. This is the key distinction that keeps Ward legally clean and architecturally sound.

Revenue Sources

- **White-label licensing** — institutions that integrate the Ward specification pay a licensing fee for SDK access, conformance support, and API access
-

- **Enterprise API licensing** — institutions running Ward at production scale license the hosted API at

api.wardprotocol.org

- **Conformance certification** — institutions seeking to market their vault as Ward-conformant pay for audit and certification

- **Integration consulting** — engineering support for first-deployment institutional partners

No Token Required

Ward Protocol does not require its own token. Revenue is denominated in USD licensing fees. This eliminates regulatory complexity, avoids securities law exposure, and keeps Ward focused entirely on real utility.

Market context: Approximately 12 people have ever used the XLS-66 lending protocol on XRPL. Deep XRPL DeFi expertise is extraordinarily rare. Ward Protocol's first-mover position as the only team building default protection infrastructure in this space is a durable competitive advantage — validated directly by the XRPL Foundation.

SECTION 09

Proof of Implementation

Ward Protocol is not a whitepaper project. It is testnet-proven infrastructure with a live SDK, live API, and confirmed on-chain transactions.

METRIC	VALUE
--------	-------

SDK Version	0.1.1 – live on PyPI
Unit Tests	75/75 passing
On-Chain Transactions	5 confirmed – XRPL Altnet, March 1 2026
External Dependencies	Zero – pure XRPL
Ward Holds Keys	Never
Authoritative State	XRPL Ledger only
License	MIT – open source

Confirmed Testnet Transactions – XRPL Altnet

STEP	TYPE	TRANSACTION HASH
1	Payment	D541B6A2156E4BB3B22D9BD1D451598DF2D0387A25B73A5918A8779D76783169
2	NFTokenMint	B323815A6C7BA98935D2C2AA3CFC94BB956E59BA716A59430F2183D2AE148C DF
3	EscrowCreate	9BB570DBC6CB9EB11339FBBDA4920E03EC2CC49EC547CBF0D031C8AABC48 B0A3
4	EscrowFinish	E65C35A568AE93E6D8A628F36A217DACB1B2A7E1A8F0A7B0912E510AED0A3D BB
5	NFTokenBurn	A5A0652C4DA629F0D46D2A3504FDC22E410848AF5D27E956E3997346A7B464 D8

Phase 1 – Protocol Specification (Completed)

- SDK built and tested — [ward_client.py](#) , 75/75 tests passing
- Testnet simulation confirmed — 5 on-chain transactions on XRPL Altnet
- XRPLF Discussion #474 — active community engagement
- PyPI package published — [ward-protocol v0.1.1](#)
- Production API live — [api.wardprotocol.org](#)
- XRPLF intro call — Executive Director validated problem and off-ledger approach

Phase 2 – Grant Phase (Q2-Q3 2026)

- Security audit — recognized XRPL-familiar audit firm
- Legal opinion — written determination that Ward is a software protocol, not an insurer
- Ward Protocol white paper — this document, published publicly
- Live risk dashboard — [dashboard.wardprotocol.org](#) , vault health monitoring
- First institutional integration — white-label licensing with first XRPL vault operator
- EVM sidechain and Flare evaluation — programmable insurance logic paths

Phase 3 – Institutional Infrastructure (2027)

- Live institutional partners with demonstrable transaction volume

- Ward Protocol established as the de facto default protection standard for XRPL DeFi
- Formal XLS standards PR – resubmit with team, audit, and community consensus
- v2 privacy layer – XLS-96 confidential balances + MPT selective disclosure

SECTION 11

Conclusion

Ward Protocol solves a specific, documented gap in XRPL infrastructure. XLS-66 lending vaults are live. Institutional interest in XRPL DeFi is growing. But without a standard default protection layer, depositors have no trustless recourse and institutions cannot credibly offer coverage.

Ward introduces that layer – deterministic, auditable, XRPL-native, and off-ledger by design. The code is written, tested, deployed, and validated by the XRPL Foundation's Executive Director.

The window to establish first-mover positioning in this space is now. Ward Protocol is the only team building here. The spec is open. The rails are yours.

Get started: `pip install ward-protocol` · wardprotocol.org · api.wardprotocol.org · [GitHub](https://github.com)

WARD Protocol

wardprotocol.org api.wardprotocol.org [dashboard](#) [github](#) XRPLF #474

MIT License · Open Source · Not an insurance company